



Web Fundamentals

WEF1VO

Teil 5: Layout-Frameworks

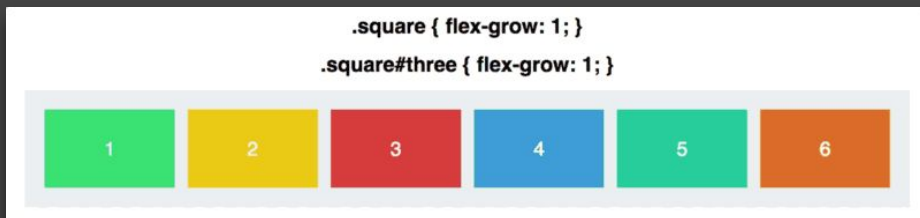
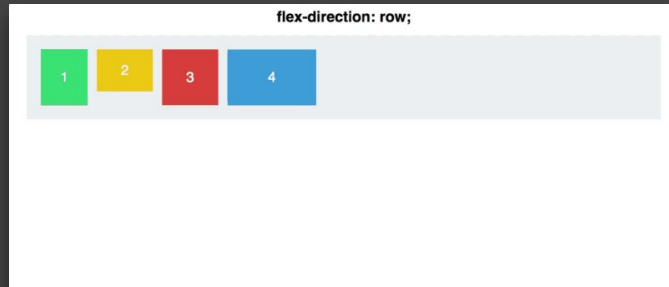
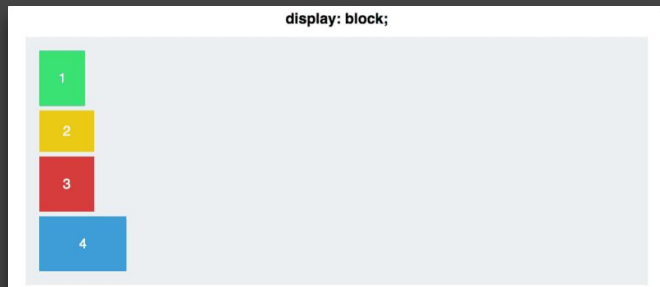
Medientechnik und -design | WS 2025/26

CSS Flexible Box Layout

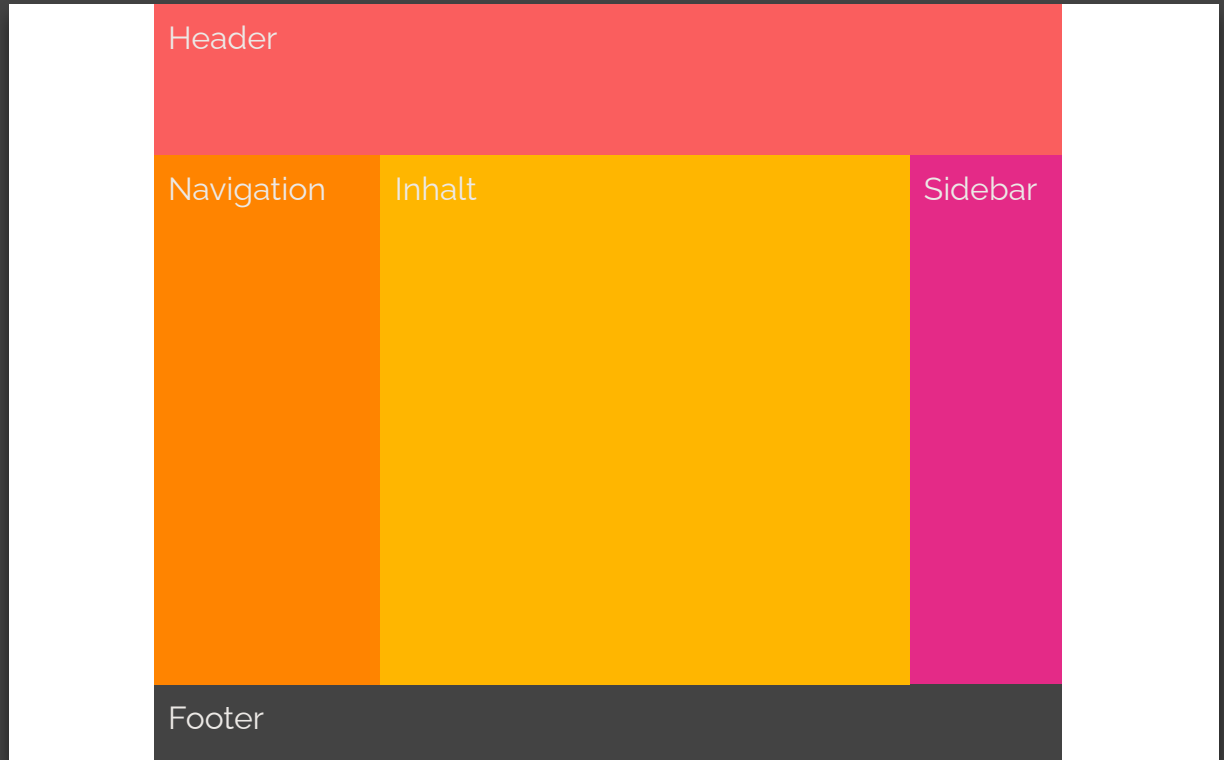
Elemente auf der Seite anordnen



Flexible Boxen



Layout s mit Flexbo x



Flexibler Container



```
<body>
  <div id="container">
    <header>
      <h1>Die Überschrift</h1>
    </header>
    <main>
      <section id="content">Lorem ipsum...</section>
      <nav>
        <ul>
          <li>Navigation 1</li>
          <li>Navigation 2</li>
        </ul>
      </nav>
      <aside>Lorem ipsum...</aside>
    </main>
    <footer>&copy; 2025 Der*Die Autor*in</footer>
  </div>
</body>
```

```
* {
  margin: 0;
  box-sizing: border-box;
}

body {
  font-family: Raleway, Arial, sans-serif;
  color: rgb(236 233 230);
  line-height: 2rem;
  hyphens: auto; /* Silbentrennung */
}

div#container {
  width: 960px;
  margin: 0 auto;
  background-color: rgb(255 132 0);
}

main {
  display: flex;
  flex-direction: row;
  min-height: 25rem;
}
```

Flexibilität & Reihenfolge



```
<body>
  <div id="container">
    <header>
      <h1>Die Überschrift</h1>
    </header>
    <main>
      <section id="content">Lorem ipsum...</section>
      <nav>
        <ul>
          <li>Navigation 1</li>
          <li>Navigation 2</li>
        </ul>
      </nav>
      <aside>Lorem ipsum...</aside>
    </main>
    <footer>&copy; 2025 Der*Die Autor*in</footer>
  </div>
</body>
```

```
main > section {
  flex: 2 2 600px; /* 62,5 % von 960px */
  order: 2;
  color: rgb(67 67 67);
  background-color: rgb(255 182 0);
}
```

```
main > nav {
  flex: 1 1 240px; /* 25 % von 960px */
  order: 1;
  background-color: rgb(255 132 0);
}
```

```
main > aside {
  flex: 1 1 120px; /* 12,5 % von 960px */
  order: 3;
  background-color: rgb(228 42 135);
}
```

Wachsen & Schrumpfen



```
@media all and (min-width: 1200px) {  
  div#container {  
    width: 1200px;  
  }  
}
```

1200px: 240px mehr Platz

- <section>: 600px, flex-grow: 2
- <nav>: 240px, flex-grow: 1
- <aside>: 120px, flex-grow: 1

$$\text{width}_{\text{new}} = (\text{flex-grow} / \text{flex-grow}_{\text{total}}) * \text{free-space} * \text{width}$$

- <section>: $(2 / 4) * 240 * 600 = 720$
- <nav>: $(1 / 4) * 240 * 240 = 300$
- <aside>: $(1 / 4) * 240 * 120 = 180$

```
@media all and (max-width: 960px) {  
  div#container {  
    width: 100%;  
  }  
}
```

720px: 240px weniger Platz

$$\text{shrink-ratio} = (\text{width} * \text{flex-shrink}) / (\text{shrink-scaled-width}_{\text{total}})$$

- <section>: 600px, flex-shrink: 2, shrink-ratio: 0,7693
- <nav>: 240px, flex-shrink: 1, shrink-ratio: 0,1538
- <aside>: 120px, flex-shrink: 1, shrink-ratio: 0,0769

$$\text{width}_{\text{new}} = \text{width} - (\text{shrink-space} * \text{shrink-ratio})$$

- <section>: $600 - (240 * 0,7693) = 414,38$
- <nav>: $240 - (240 * 0,1538) = 203,08$
- <aside>: $120 - (240 * 0,0769) = 101,54$

$$600 * 2 + 240 * 1 + 120 * 1 = 1560$$

Ausrichtung der Boxen



```
<body>
  <div id="container">
    <header>
      <h1>Die Überschrift</h1>
    </header>
    <main>
      <section id="content">Lorem ipsum...</section>
      <nav>
        <ul>
          <li>Navigation 1</li>
          <li>Navigation 2</li>
        </ul>
      </nav>
      <aside>Lorem ipsum...</aside>
    </main>
    <footer>&copy; 2025 Der*Die Autor*in</footer>
  </div>
</body>
```

```
header {
  display: flex;
  align-items: center;
  background-color: rgb(250 94 94);
}
```

```
footer {
  display: flex;
  align-items: flex-end;
  justify-content: flex-end;
  background-color: rgb(67 67 67);
}
```

```
header, footer {
  min-height: 6rem;
}
```


Flex **responsively**



```
@media all and (max-width: 960px) {  
  div#container {  
    width: 100%;  
  }  
}
```

```
@media all and (max-width: 640px) {  
  main {  
    flex-direction: column;  
  }  
  main > section, main > nav, main > aside {  
    order: 0;  
    flex-basis: auto;  
  }  
}
```

CSS Grid Layout

Objekte am zweidimensionalen Raster anordnen

Warum denn



Flexbox
eindimensional



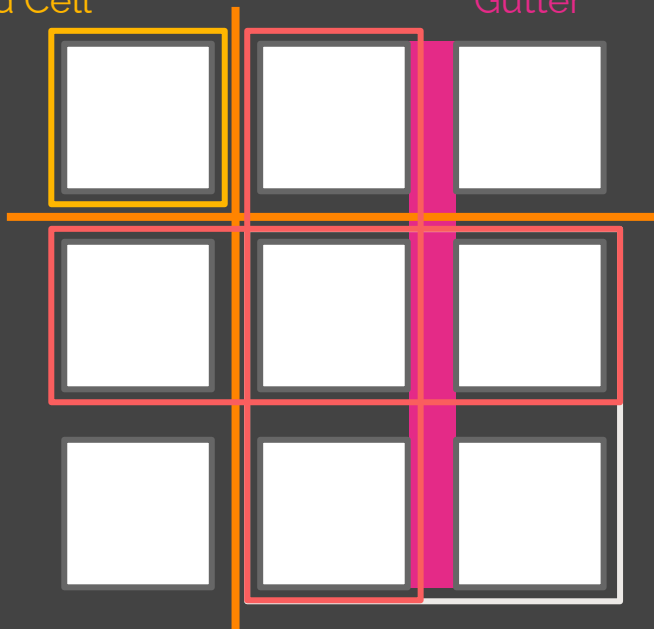
CSS Grid
zweidimensional

Grid Terminologie



Grid Cell

Gutter



Grid Lines

Grid Row

Grid Column

Grid Area

Grid Track

Allgemein für Grid Row oder Grid Column

Grid Container

HTML-Element mit Grid Eigenschaft

Grid Items

HTML-Elemente im Container

Grid mit Autoplacement



<p>Der folgende Container nimmt standardmäßig die volle verfügbare Breite ein:</p>

```
<div class="container block-grid">  
  <div class="item">A</div>  
  <div class="item">B</div>  
</div>
```

<p>Dadurch wird der Textfluss vor und nach dem Grid unterbrochen.</p>

```
.block-grid {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  gap: 0.625rem;  
}  
.item {  
  background-color: rgb(255 204 0);  
  padding: 0.625rem;  
  text-align: center;  
  font-weight: bold;  
}
```



Inline-Grid (Autoplacem.)

```
<p>
  Hier ist der "Platz" für ein Grid, das sich
  befindet sich
  <span class="container inline-grid">
    <span class="item">X</span>
    <span class="item">Y</span>
  </span>
  ein Inline-Grid. Es fließt mit dem Text und
  erzwingt keinen Umbruch.
  Dank <code>vertical-align: middle</code>
  sitzt es mittig auf der Zeile.
</p>
```

```
.inline-grid {
  display: inline-grid;
  grid-template-columns: auto auto;
  gap: 0.4rem;
  vertical-align: middle;
  padding: 0.4rem;
}

.inline-grid .item {
  padding: 0.1rem 0.3rem;
  font-size: 0.9em;
}
```

Subgrid



(Autoplacement)

```
<div class="container main-grid">  
  <div class="item">1</div>
```

```
  <div class="subgrid-container">  
    <div class="sub-item">Sub A</div>  
    <div class="sub-item">Sub B</div>  
    <div class="sub-item">Sub C</div>  
  </div>
```

```
  <div class="item">5</div>  
</div>
```

```
.main-grid {  
  display: grid;  
  grid-template-columns: repeat(5, 1fr);  
  gap: 0.625rem;  
}
```

```
.subgrid-container {  
  grid-column: span 3;  
  display: grid;  
  grid-template-columns: subgrid;  
  background-color: rgb(224 224 224);  
  border: 0.2rem dashed rgb(102 102 102);  
  padding: 0;  
}
```

```
.sub-item {  
  background-color: rgb(255 85 85);  
  color: white;  
  padding: 0.625rem;  
  text-align: center;  
  font-weight: bold;  
}
```

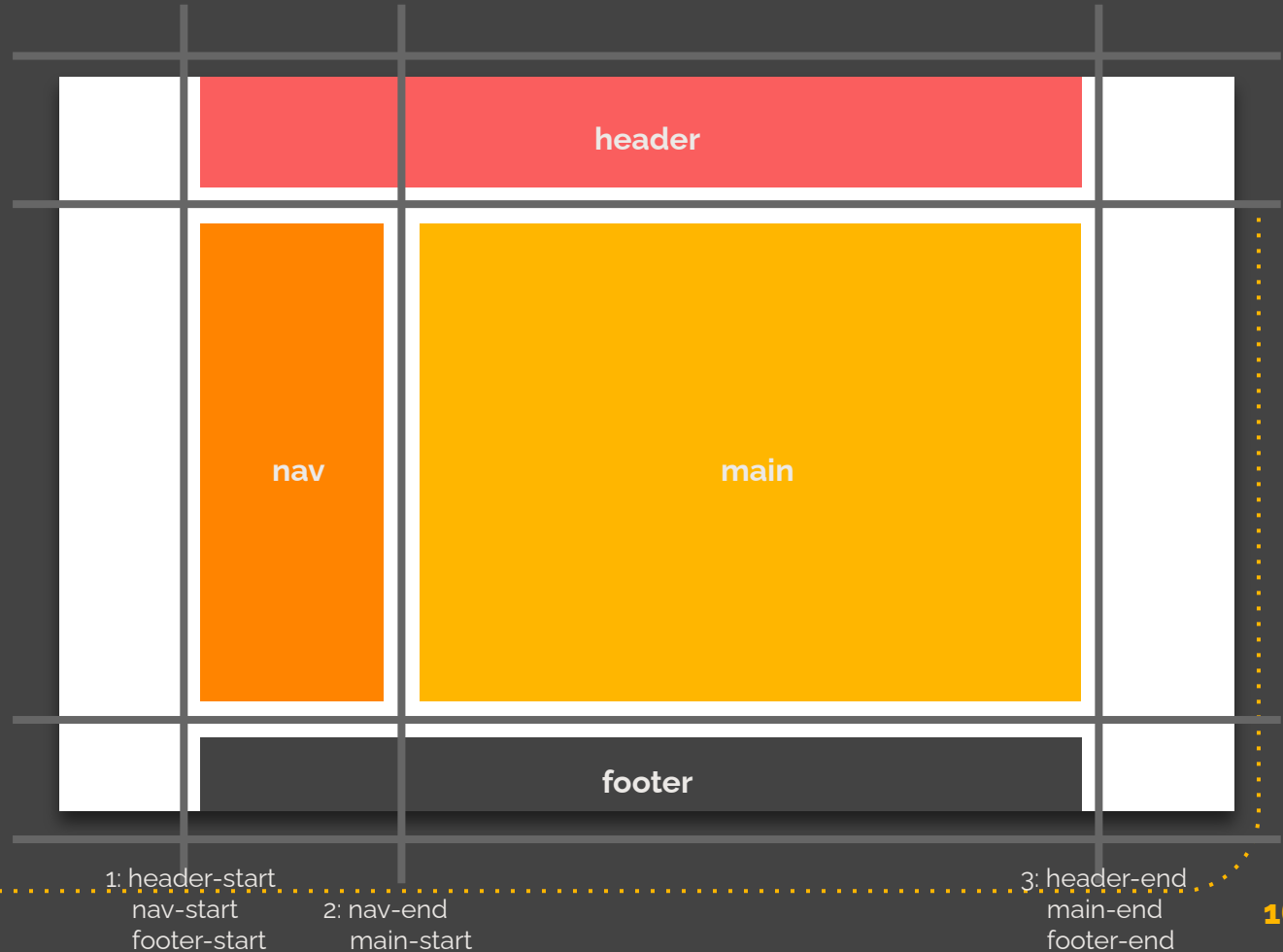
Grid im Layout

1: row1-start

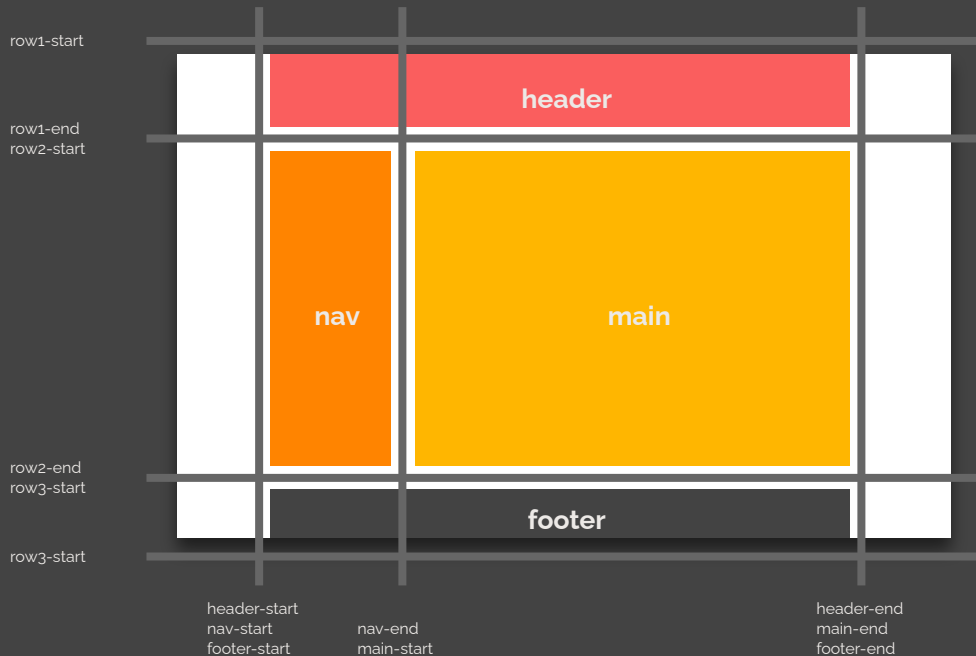
2: row1-end
row2-start

3: row2-end
row3-start

4: row3-end

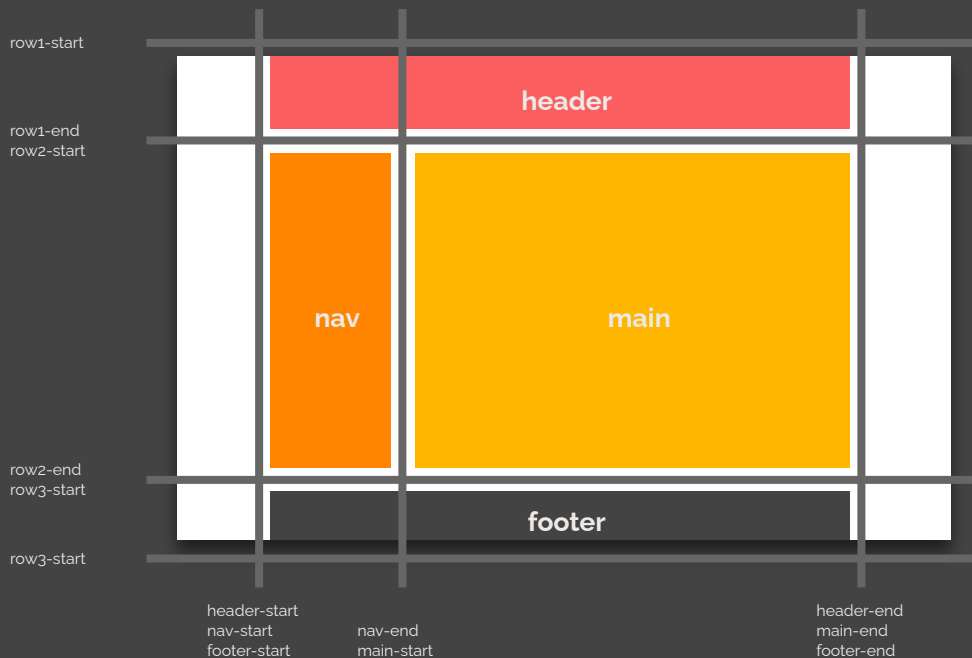


Grid Container



```
div#container {  
  display: grid;  
  max-width: 75rem;  
  margin: 0 auto;  
  
  grid-template-columns:  
    [header-start nav-start footer-start] 1fr  
    [nav-end main-start] 3fr  
    [header-end main-end footer-end];  
  
  grid-template-rows:  
    [row1-start] 6rem  
    [row1-end row2-start] 1fr  
    [row2-end row3-start] 6rem  
    [row3-end];  
  
  grid-template-areas:  
    "header header"  
    "nav main"  
    "footer footer";  
  
  gap: 0.5rem;  
}
```

Elemente zuordnen



```
header {  
  grid-column: header-start / header-end;  
  grid-row: row1-start / row1-end;  
  grid-area: header;  
}  
  
nav {  
  grid-column: nav-start / nav-end;  
  grid-row: row2-start / row2-end;  
  grid-area: nav;  
}  
  
main {  
  grid-column: main-start / main-end;  
  grid-row: row2-start / row2-end;  
  grid-area: main;  
}  
  
footer {  
  grid-column: footer-start / footer-end;  
  grid-row: row3-start / row3-end;  
  grid-area: footer;  
}
```

Responsive Grid



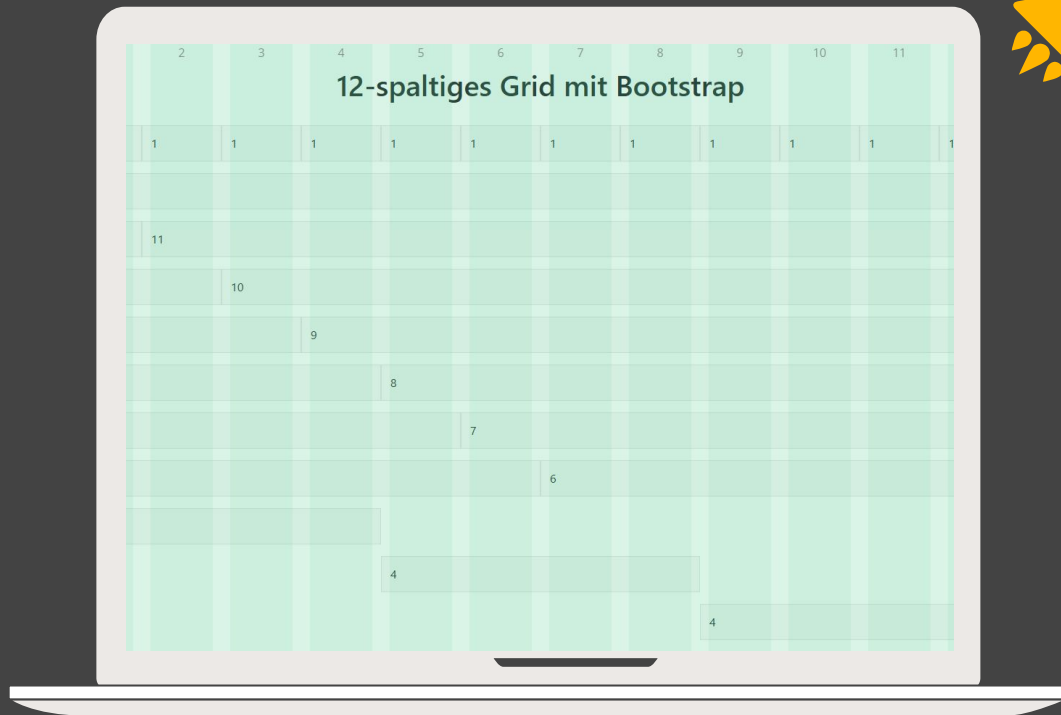
```
@media all and (max-width: 640px) {  
  div#container {  
    grid-template-rows:  
      [row1-start] 6rem  
      [row1-end row2-start] auto  
      [row2-end row3-start] auto  
      [row3-end row4-start] 6rem  
      [row4-end];  
  }  
  nav {  
    grid-column: nav-start / header-end;  
  }  
  main {  
    grid-column: header-start / main-end;  
    grid-row: row3-start / row3-end;  
  }  
  footer {  
    grid-row: row4-start / row4-end;  
  }  
}
```

Bootstrap

Ein modernes CSS-Framework

Raster mit 12 Spalten

Inhalte können zwischen
1 und 12 Spalten
aufgeteilt werden





Bootstrap

5.3.38

Responsive, mobile-first
CSS-Framework

Anordnung der Elemente mit
Flexbox

Set von CSS-Stilen und
JavaScript-Plugins



<https://getbootstrap.com/>

Bootstrap einbinden

```
<link rel="stylesheet" href="css/bootstrap.css">
```

VS.

```
<link rel="stylesheet" href="css/bootstrap.min.css">
```

VS.

```
<link rel="stylesheet" href="css/bootstrap-grid.css">
```

VS.

```
<link  
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"  
  rel="stylesheet"  
  integrity="sha384-sRIL4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"  
  crossorigin="anonymous">
```

Download

[View on GitHub](#)

Download Bootstrap to get the compiled CSS and JavaScript, source code, or include it with your favorite package managers like npm, RubyGems, and more.

Compiled CSS and JS

Download ready-to-use compiled code for **Bootstrap v5.3.8** to easily drop into your project, which includes:

- Compiled and minified CSS bundles (see [CSS files comparison](#))
- Compiled and minified JavaScript plugins (see [JS files comparison](#))

This doesn't include documentation, source files, or any optional JavaScript dependencies like Popper.

[Download](#)



Bootstrap Container

	X-Small < 576px	Small ≥ 576px	Medium ≥ 768px	Large ≥ 992px	X-Large ≥ 1200px	XX-Large ≥ 1400px
.container	100%	540px	720px	960px	1140px	1320px
.container-sm	100%	540px	720px	960px	1140px	1320px
.container-md	100%	100%	720px	960px	1140px	1320px
.container-lg	100%	100%	100%	960px	1140px	1320px
.container-xl	100%	100%	100%	100%	1140px	1320px
.container-xxl	100%	100%	100%	100%	100%	1320px
.container-fluid	100%	100%	100%	100%	100%	100%



Bootstrap Spalten

	X-Small < 576px	Small ≥ 576px	Medium ≥ 768px	Large ≥ 992px	X-Large ≥ 1200px	XX-Large ≥ 1400px
Containerbreite	Auto	540px	720px	960px	1140px	1320px
Klassenpräfix	.col-	.col-sm-	.col-md-	.col-lg-	.col-xl-	.col-xxl-

```
<div class="container">
  <div class="row">
    <div class="col-6">Spalte 1</div>
    <div class="col-6">Spalte 2</div>
  </div>
</div>
```

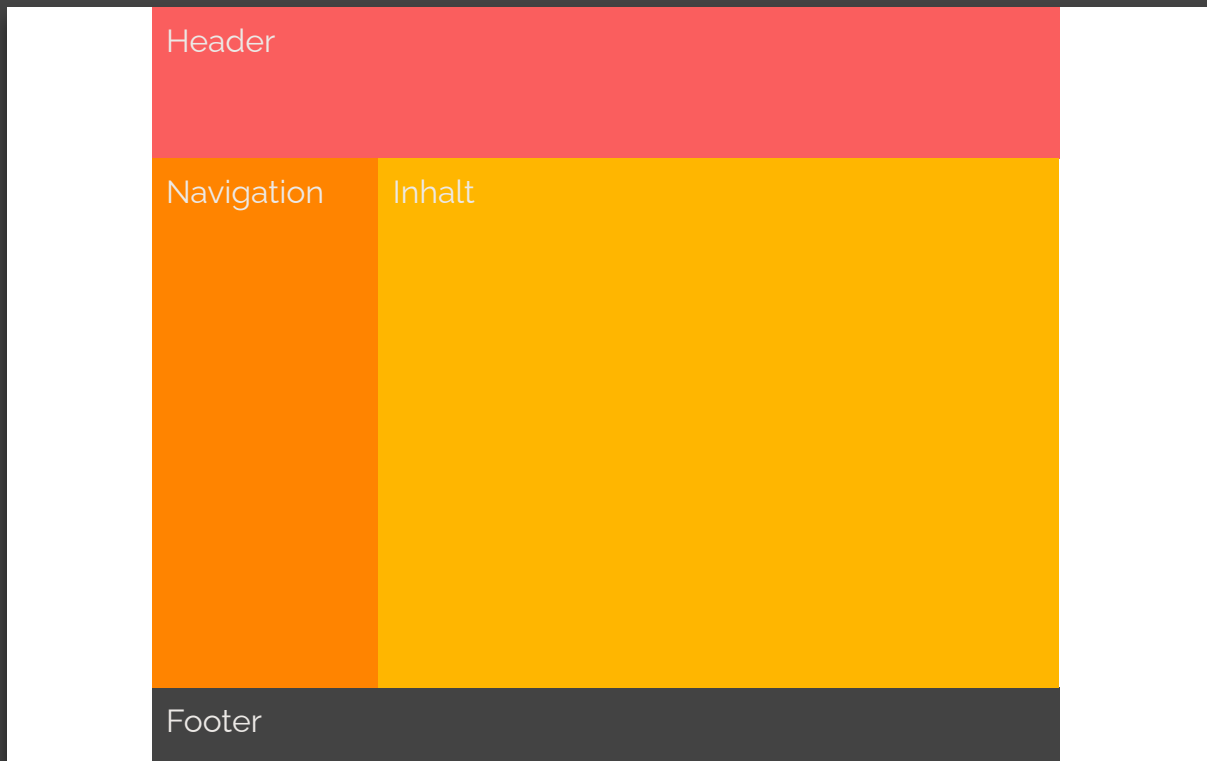
Spalten responsive



```
<div class="container">
  <div class="row">
    <div class="col-md-4 col-lg-3">Spalte 1</div>
    <div class="col-md-8 col-lg-6">Spalte 2</div>
    <div class="col-md-12 col-lg-3">Spalte 3</div>
  </div>
</div>
```

```
<div class="container">
  <div class="row">
    <div class="col-md-6 col-xl-4 col-xxl-3">Sp. 1</div>
    <div class="col-md-6 col-xl-4 col-xxl-3">Sp. 2</div>
    <div class="col-md-6 col-xl-4 col-xxl-3">Sp. 3</div>
    <div class="col-md-6 col-xl-4 col-xxl-3">Sp. 4</div>
    <div class="col-md-6 col-xl-4 col-xxl-3">Sp. 5</div>
    <div class="col-md-6 col-xl-4 col-xxl-3">Sp. 6</div>
    <div class="col-md-6 col-xl-4 col-xxl-3">Sp. 7</div>
    <div class="col-md-6 col-xl-4 col-xxl-3">Sp. 8</div>
    <div class="col-md-6 col-xl-4 col-xxl-3">Sp. 9</div>
    <div class="col-md-6 col-xl-4 col-xxl-3">Sp. 10</div>
    <div class="col-md-6 col-xl-4 col-xxl-3">Sp. 11</div>
    <div class="col-md-6 col-xl-4 col-xxl-3">Sp. 12</div>
  </div>
</div>
```

Layout mit Bootstrap



Das Markup



```
<body>
  <div class="container">
    <div class="row">
      <header class="col-md-12">
        <h1>Überschrift</h1>
      </header>
      <nav class="col-md-3">
        <ul>
          <li>Navigation 1</li>
          <li>Navigation 2</li>
        </ul>
      </nav>
      <main class="col-md-9">Lorem ipsum...</main>
      <footer class="col-md-12">&copy; 2025 Der*Die Autor*in</footer>
    </div>
  </div>
</body>
```



There's **more** where that came

from

Bootstrap, CSS
Utility-Klassen (Gutter,
Spacing etc.)

Formulare

Bilder

uvm.

Bootstrap JavaScript

JavaScript Plugins für
viele UI-Elemente

Carousel, Accordion,
Tabs etc.

CSS-Frameworks

960.gs (Urvater)

Unsemantic

Foundation

Tailwind

uvm.

CSS Custom Properties

CSS Variablen

Das alte Problem



```
.one {  
  color: white;  
  background-color: rgb(255 182 0);  
  margin: 10px;  
  width: 50px;  
  height: 50px;  
}  
  
.two {  
  color: white;  
  background-color: rgb(255 182 0);  
  margin: 10px;  
  width: 150px;  
  height: 70px;  
}  
  
.three {  
  color: white;  
  background-color: rgb(255 182 0);  
  margin: 10px;  
  width: 75px;  
  height: 75px;  
}
```

CSS Custom Properties



```
body {  
  --text-color: rgb(236 233 230);  
  color: var(--text-color);  
}
```




Custom Properties

Anlegen

Immer in einer CSS-Datei

Beginnt mit `--`.

```
body {  
  --bg-color: rgb(255 182 0);  
}  
  
:root {  
  --bg-color: rgb(255 182 0);  
}  
  
p {  
  --text-color: rgb(67 67 67);  
}
```

Anwenden

Mit `var()` Funktion und Name des Properties. Fallback möglich.

```
div, p {  
  background-color: var(--bg-color);  
}  
  
div, p {  
  background-color: var(--bg-color, red);  
}
```

```
body {
  --bg-color: rgb(255 182 0);
  --text-color: rgb(236 233 230);
  --margin: 10px;
  --size: 200px;
  font-family: Raleway, Arial, sans-serif;
  color: var(--text-color);
  display: flex;
}

div, p {
  width: var(--size);
  height: var(--size);
  margin: var(--margin);
  background-color: var(--bg-color, red);
  display: flex;
  justify-content: center;
  align-items: center;
}

div {
  --bg-color: rgb(250 94 94);
}

p {
  --text-color: rgb(67 67 67);
  color: var(--text-color);
}
```

Die Kaskade nützen



CSS Reset & Normalisierung

Browserunterschiede vereinheitlichen

CSS Reset



Einfach

```
* {  
  margin: 0;  
}
```

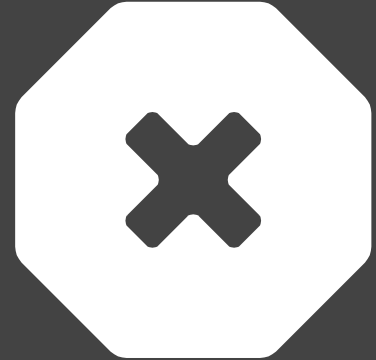
Besser?

```
* {  
  padding: 0;  
  margin: 0;  
  border: 0;  
}
```

Noch besser?

```
html, body {  
  padding: 0;  
  margin: 0;  
}  
  
html {  
  font-size: 1em;  
}  
  
body {  
  font-size: 100%;  
}  
  
a img, :link img, :visited img {  
  border: 0;  
}
```

Eric Meyers Reset CSS



<https://meyerweb.com/eric/tools/css/reset/index.html>



CSS Normalisierung

Sinnvolle Styles beibehalten

Große Anzahl an Elementen normalisieren

Fehler beheben

Usability beheben

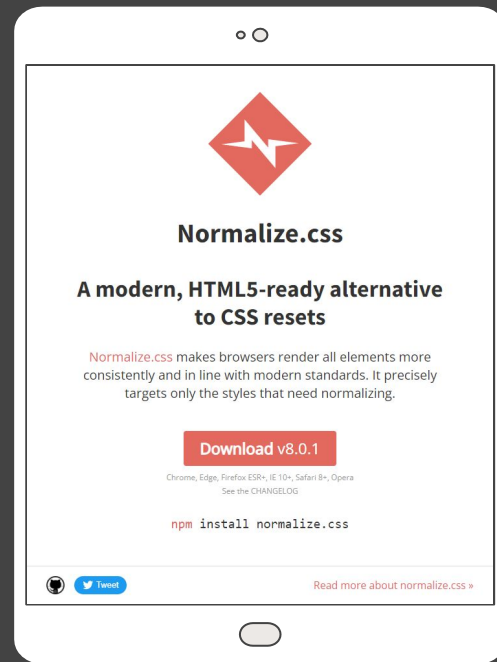
Ursprung:

<https://necolas.github.io/normalize.css/>

Moderne Varianten:

<https://github.com/csstools/normalize.css/>

<https://github.com/sindresorhus/modern-normalize/>



CSS Methodiken

CSS-Stile organisieren



Historische Ansätze

Object Oriented **CSS**

Wiederverwendbare
Objekte erzeugen:
Struktur und Aussehen
trennen.

Scalable and Modular Architecture for **CSS**

Regeln in Kategorien wie
Base, Layout, Module und
State einteilen.

Inverted Triangle **CSS**

CSS in Schichten einteilen.
Z.B. Settings -> Tools ->
Generic -> Elements ->
Objects -> Components ->
Utilities

BEM

Block Element Modifier



Block



CSS

```
.site-search {  
  ...  
}
```

HTML

```
<form action="..." class="site-search">  
  ...  
</form>
```

Element



CSS

```
.site-search__field {  
  ...  
}
```

HTML

```
<form action="..." class="site-search">  
  <input type="search" class="site-search__field">  
  <button type="submit" class="site-search__button">  
    Search  
  </button>  
</form>
```

Modifier



CSS

```
.site-search-full {  
  ...  
}  
  
.site-search__button--clicked {  
  ...  
}
```

HTML

```
<form action="..." class="site-search site-search--full">  
  <input type="search" class="site-search__field">  
  <button type="submit" class="site-search__button">  
    Search  
  </button>  
</form>
```

SUIT CSS

Style Tools for UI Components



Components



CSS

```
.SiteSearch {  
  ...  
}
```

HTML

```
<form action="..." class="SiteSearch">  
  ...  
</form>
```

Descendants



CSS

```
.SiteSearch-field {  
  ...  
}
```

HTML

```
<form action="..." class="SiteSearch">  
  <input type="search" class="SiteSearch-field">  
  <button type="submit" class="SiteSearch-button">  
    Search  
  </button>  
</form>
```

Modifier



CSS

```
.SiteSearch--full {  
  ...  
}
```

HTML

```
<form action="..." class="SiteSearch SiteSearch--full">  
  <input type="search" class="SiteSearch-field">  
  <button type="submit" class="SiteSearch-button">  
    Search  
  </button>  
</form>
```


States



CSS

```
.SiteSearch {  
  ...  
}
```

```
.SiteSearch.is-searching {  
  ...  
}
```

HTML

```
<form action="..." class="SiteSearch is-searching">  
  ...  
</form>
```

Utilities



CSS

```
.u-textCenter {  
  text-align: center;  
}
```

HTML

```
<form action="..." class="SiteSearch u-textCenter">  
  ...  
</form>
```

CASCADE LAYERS



CSS-Regeln in eigenen Schichten verwalten

Oder: Nativ das erledigen, was die ersten CSS-Methodiken erreichen wollten.

Layer definieren



```
@layer reset, base, components, utilities;
```

Definiert 4 Layer.

Je weiter hinten desto wichtiger.

Regeln an Layer



zuweisen

```
@layer reset {
  *, *::before, *::after {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
  }
}

@layer base {
  body {
    font-family: system-ui, sans-serif;
    line-height: 1.5;
  }
}

@layer components {
  .button {
    background: steelblue;
    color: white;
  }
}

@layer utilities {
  .text-center {
    text-align: center;
  }
}
```

```
@import url("modern-normalize.css") layer(reset);
@import url("base.css") layer(base);
@import url("components.css") layer(components);
@import url("utilities.css") layer(utilities);
```

CSS People

Whom to follow?



Kevin Powell

CSS Evangelist aus Kanada.
Videos, Shorts und Kurse zu allem
rund um CSS.

<https://www.kevinpowell.co/>

<https://www.youtube.com/kevinpowell>





Una Kravets

Web Developerin im Google Chrome Team. Host des CSS Podcast (mit Bramus Van Damme).

<https://una.im/>

<https://pod.link/thecsspodcast>





And many more

- [Rachel Andrew](#)
- [Adam Argyle](#)
- [Andy Bell](#) ([Piccalilli](#))
- [Chris Coyier](#) ([Codepen](#) | [CSS Tricks](#))
- [Stephanie Eckles](#) ([Modern CSS](#) | [Smol CSS](#))



Nächstes Mal in WEF1VO

Web Accessibility

Credits



- S. 3: Flexbox in GIFs: <https://www.freecodecamp.org/news/an-animated-guide-to-flexbox-d280cf6afc35/>
- S. 3: Flexbox in GIFs: <https://www.freecodecamp.org/news/even-more-about-how-flexbox-works-explained-in-big-colorful-animated-gifs-a5a74812b053/>
- S. 19: Bootstrap Logo: <https://getbootstrap.com/docs/5.3/about/brand/>
- S. 20: Bootstrap Webpage: <https://getbootstrap.com/docs/5.3/getting-started/download/>
- S. 35: Normalize.css Webpage: <https://necolas.github.io/normalize.css/>
- S. 55: Kevin Powell bei Frontend Masters: <https://frontendmasters.com/teachers/kevin-powell/>
- S. 56: Una Kravets: <https://una.im/about>